ITC3304
WEB APPLICATION DEVELOPMENT

Module 2: Control Structures

Instructor: Ahmad Tijjani Garba

1- INTRODUCTION

1.0 Learning Objectives:

At the end of this Module, the student will be able to understand the following:

- Control Structures in PHP
- Conditional Statements in PHP
- Looping Structures in PHP
- Use of Logical Operators in PHP
- Best Practices for Code Organization and Readability in PHP

1.1 Control Structures in PHP

- Control structures in are an essential part of any programming language, including PHP.
- They are used to control the flow of a program by allowing the programmer to specify which statements should be executed under certain conditions.
- Understanding control structures in PHP is crucial for any aspiring PHP developer, as they form the backbone of many programming tasks.
- In PHP, there are two primary types of Control Structures:
  - Conditional Statements and;
  - Control Loops.

PART 1

CONDITIONAL STATEMENTS

1.2 CONDITIONAL STATEMENTS

1.2.1: The PHP If Statement:

- The PHP If Statement is a control structure in PHP that allows you to execute code based on a specific condition.
- It is one of the most commonly used control structures in PHP The If Statement evaluates a condition and executes a block of code only if the condition is true.
- The basic syntax for the If Statement is as follows:

```
if (condition) {
    // code to execute if the condition is true
}
```

1.2 CONDITIONAL STATEMENTS....

1.2.1: The PHP If Statement Example:

- Here's an example of how the If Statement works in PHP:

```php
<?php
$number = 10;

if ($number > 5) {
    echo "The number is greater than 5";
}
?>
```

- Explanation: In this example, we have assigned a value of 10 to the number variable. The If Statement then checks if the value of the number variable. The If Statement then checks if the value of the number is greater than 5. Since 10 is greater than 5, the condition is true and the code inside the curly braces { } is executed.

- Run the above code in your editor for a better and clear explanation.

1.2 CONDITIONAL STATEMENTS….

1.2.2: The PHP Else Statement:

- In PHP, the else statement (control structures in PHP) is used in conjunction with an if statement to execute code if the if condition is false. The syntax for an if-else statement is as follows:

```
if (condition) {
  // code to be executed if the condition is true
} else {
  // code to be executed if the condition is false
}
```

1.2 CONDITIONAL STATEMENTS….

1.2.1: The PHP Else Statement Example:

- Here's an example to illustrate the use of an if-else statement in PHP:

```php
<?php
$age = 18;

if ($age >= 18) {
  echo "You are eligible to vote.";
} else {
  echo "You are not eligible to vote.";
}
?>
```

- In the above example, we have a variable age assigned a value of 18. The if statement checks whether age assigned a value of 18. The if statement checks whether the age is greater than or equal to 18. Since $age is 18, the condition evaluates to true, and the code inside the if block is executed, which outputs "You are eligible to vote."

1.2 CONDITIONAL STATEMENTS….

1.2.1: The PHP Else Statement Example:

```php
<?php
$age = 18;

if ($age >= 18) {
  echo "You are eligible to vote.";
} else {
  echo "You are not eligible to vote.";
}
?>
```

- If the condition had been evaluated as false, the code inside the else block would have been executed instead, outputting "You are not eligible to vote." It's important to note that the else statement is optional in an if statement.

- Run the above code in your editor for a better and clear explanation.

1.2 CONDITIONAL STATEMENTS….

1.2.2: The PHP Else-If Statement:

- In PHP, the else if statement (control structures in php) allows you to test multiple conditions and execute different code blocks based on the results. The else-if statement is often used when you need to check for more than two conditions. Here is an example of how to use the else if statement in PHP:

```
if(condition 1){
    //do something
}else if(condition 2){
    //do something
}else if(condition ){
    //do something
}else{
    //do something
}
```

1.2 CONDITIONAL STATEMENTS….

1.2.2: The PHP Else-If Statement Example:

```php
<?php
$score = 75;

if ($score >= 90) {
    echo "Your grade is A";
} else if ($score >= 80) {
    echo "Your grade is B";
} else if ($score >= 70) {
    echo "Your grade is C";
} else if ($score >= 60) {
    echo "Your grade is D";
} else {
    echo "Your grade is F";
}
?>
```

- **Explanation:** In this example, we have a variable score with a value of '75'. We are using the if statement to test the value of the score with a value of '75'. We are using the if statement to test the value of the score against different conditions using the else if statement.

1.2 CONDITIONAL STATEMENTS….

1.2.2: The PHP Else-If Statement Example:

- If the value of the $score is greater than or equal to 90, then the code block inside the first if statement will be executed, and the message "Your grade is A" will be displayed.
- If the value of the score is not greater than or equal to ‘90‘, the script will move on to the next else if statement. If the value of score is not greater than or equal to ‘90‘, the script will move on to the next else if statement. If the value of score is greater than or equal to 80, then the code block inside the second else if statement will be executed, and the message "Your grade is B" will be displayed. This process continues for each else if statement until a condition is met.
- Run the above code in your editor for a better and clear explanation.

1.2 CONDITIONAL STATEMENTS….

1.2.3: The PHP Switch Statement:

- In PHP, a switch statement provides a way to execute different blocks of code based on the value of a variable or expression. It's similar to an if/else statement but can be easier to read and write when dealing with multiple conditions.

- The basic **syntax** of a switch statement is as follows:

```
switch (expression) {
    case value1:
        // code to execute if expression equals value1
        break;
    case value2:
        // code to execute if expression equals value2
        break;
    // add as many cases as needed
    default:
        // code to execute if none of the cases match
        break;
}
```

1.2 CONDITIONAL STATEMENTS….

1.2.3: The PHP Switch Statement Example:

- Let us see an example of a switch statement in PHP:

```php
$color = "red";

switch ($color) {
    case "red":
        echo "The color is red.";
        break;
    case "green":
        echo "The color is green.";
        break;
    case "blue":
        echo "The color is blue.";
        break;
    default:
        echo "The color is unknown.";
        break;
}
```

- **Explanation:**
  - In this example, we start with the switch keyword followed by the expression we want to test, which is color in this case. Then we have several case statements, each of which tests a specific value. If the value of color in this case. Then we have several case statements, each of which tests a specific value. If the value of color matches one of the case values, then the corresponding code block is executed. If none of the case values match, then the default block is executed.

1.2 CONDITIONAL STATEMENTS….

1.2.3: The PHP Switch Statement Example:

- So, in this example, since the value of color is "red", the first case block is executed, which outputs “The color is red.“ If color is "red", the first case block is executed, which outputs "The color is red.“ If color had been "green" or "blue", then the corresponding case block would have been executed instead.
- Run the above code in your editor for a better and clear explanation.
- A switch statement is a useful tool for executing different blocks of code based on the value of a variable or expression. It can make code easier to read and write when dealing with multiple conditions, and it's a common feature in many programming languages, including PHP.

PART 2

CONTROL LOOPS IN PHP

1.3 CONTROL LOOPS

1.3.0: INTRODUCTION:

- There are instances where you need to perform repetitive tasks, such as formatting data pulled from a database, sending out emails to a mailing list, or cycling through the contents of an array. Control Loops allow you to perform these tasks almost effortlessly.
- In PHP, just like any other programming language, loops are used to execute the same code block for a specified number of times.
- Except for the common loop types (for, while, do. . . while), PHP also support foreach loops, which is not only specific to PHP.
- Languages like Javascript and C# already use foreach loops. Let's have a closer look at how each of the loop types works.

1.3 CONTROL LOOPS

1.3.1: For Loops :

- In PHP, a for loop is used to execute a block of code repeatedly for a specified number of times. The general syntax for a for loop in PHP is as follows:

```
for (initialization; condition; increment) {
  // code to be executed
}
```

- Let's break down the components of a for loop in PHP:
  - **Initialization:** This sets the initial value of the loop variable. It is executed only once at the beginning of the loop.
  - **Condition:** In this part of the for loop, we write the condition that is checked before each iteration of the loop. If the condition is true, the loop continues to execute. If the condition is false, the loop terminates.
  - **Increment:** In this part of the for loop the operation is performed after each iteration of the loop. It is usually used to increment or decrement the loop variable value.

1.3 CONTROL LOOPS

1.3.1: For Loops Example :

- Here's an example of a for loop that prints the numbers from 1 to 10:

```
for ($i = 1; $i <= 10; $i++) {
  echo $i . "\n";
}
```

- **Output:** The code above outputs 1-10 each in a newline before terminating.
- **Explanation** In this example, we initialize the loop variable I to1, set the condition to continue looping as long as I to1, set the condition to continue looping as long as i is less than or equal to 10, and increment I by1after each iteration. The code inside the loop simply prints the value of I by1after each iteration. The code inside the loop simply prints the value of I followed by a newline character.
- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.2: Nesting For loops:

- In PHP, you can use nested for loops to iterate through multiple arrays or perform repeated operations on multiple sets of data. Nested-for loops contain one or more inner loops inside an outer loop, where each inner loop executes its own set of instructions based on the iteration of the outer loop.

- Here is an example of nesting for loops in PHP:

```php
<?php

$array1 = array("apple", "banana", "orange");
$array2 = array("red", "yellow", "orange");

for ($i = 0; $i < count($array1); $i++) {
    for ($j = 0; $j < count($array2); $j++) {
        echo $array1[$i] . " is " . $array2[$j] . "<br>";
    }
}
?>
```

1.3 CONTROL LOOPS

1.3.2: Nesting For loops Example:

```php
<?php

$array1 = array("apple", "banana", "orange");
$array2 = array("red", "yellow", "orange");

for ($i = 0; $i < count($array1); $i++) {
    for ($j = 0; $j < count($array2); $j++) {
        echo $array1[$i] . " is " . $array2[$j] . "<br>";
    }
}
?>
```

- **Explanation:** In this example, we have defined two arrays named array1and array 1 and array2. We then use nested for loops to iterate through both arrays and print out each combination of elements. The outer loop runs for the length of array1, while the inner loop runs for the length of array1, while the inner loop runs for the length of array 2. For each iteration of the outer loop, the inner loop runs through all the elements of $array2.

- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.3: Breaking out of a Loop :

- In PHP, a loop is a programming structure that allows you to execute a block of code repeatedly based on a specified condition. There are several types of loops available in PHP, including the "for" loop, "while" loop, and "do-while" loop. Sometimes, you may need to break out of a loop prematurely, even if the loop condition has not been met yet. In such cases, you can use the "break" statement to terminate the loop and exit the loop's code block.
- Here's an example of how to use the "break" statement in PHP to break out of a loop:

1.3 CONTROL LOOPS

1.3.3: Breaking out of a Loop :

- **Example:**

```php
<?php

for ($i = 1; $i <= 10; $i++) {


    if ($i > 5) {
        break;
    }

    echo $i . "<br>";
}

echo "Loop completed!";
?>
```

- **Output:**

```
1
2
3
4
5
Loop completed!
```

- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.3: Breaking out of a Loop :

- **Explanation** In the example above, we use a "for" loop to iterate through the numbers 1 to 10. However, we add an "if" statement inside the loop to check if the current number is greater than 5. If it is, we use the "break" statement to terminate the loop and exit the code block. As a result, the loop will only execute five times, from 1 to 5, and then exit. Run the above code in your editor for a better and clear explanation.

- As you can see, the loop terminates after the number 5, and the "Loop completed!" message is displayed. This is because the "break" statement causes the program to jump out of the loop's code block and continue with the next line of code after the loop.

1.3 CONTROL LOOPS

1.3.4: Continuing to the next loop iteration :

- The "continue" statement in Php is used to skip the current iteration of a loop and continue to the next iteration or next condition. This can be useful in cases where you want to skip over certain values or conditions within the loop without ending the loop entirely.
- Here's an example of using "continue" within a "for" loop in PHP:

```php
for ($i = 0; $i < 10; $i++) {
  if ($i == 5) {
    // Skip the rest of the loop for $i = 5
    continue;
  }
  echo $i . " ";
}
```

Output

```
0 1 2 3 4 6 7 8 9
```

1.3 CONTROL LOOPS

1.3.4: Continuing to the next loop iteration example :

- **Explanation:**
- In this example, the loop will run from i=0 to i=0 to i= 9. However, when $i is equal to 5, the "continue" statement is triggered, which skips the rest of the loop code and moves on to the next iteration.
- Run the above code in your editor for a better and clear explanation.
- The "continue" statement can also be used within other types of loops in PHP, such as "while" and "foreach" loops.

1.3 CONTROL LOOPS

1.3.5: For-Each loop:

- The For-Each loop in PHP is a type of loop that is used to iterate over arrays or objects. It is also known as the "for each" loop. The For-Each loop is easier to use than the traditional For loop, especially when you are working with arrays. The syntax of the For-Each loop in PHP is as follows:

```
for each ($array as $value) {
    // Code to be executed for each element in the array
}
```

- **Explanation** In this syntax, array is the array that you want to iterate over, and array is the array that you want to iterate over, and value is the variable that will hold the value of each element in the array as you iterate over it.

- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.5: For-Each loop Example 1:

- Let's take an example of an array of fruits and use the For-Each loop to iterate over the elements of the array.

```php
$fruits = array("Apple", "Banana", "Orange", "Grapes");

for each ($fruits as $fruit) {
    echo $fruit. "<br>";
}
```

Output

```
Apple
Banana
Orange
Grapes
```

- **Explanation** In this example, we have created an array of fruits and used the For-Each loop to iterate over the elements of the array. The loop will execute once for each element in the array, and the variable fruit will hold the value of each element as it iterates over the array. The code inside the loop will then echo the value of the fruit variable and add a line break after each fruit. Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.5: For-Each loop Example 2:

- The For-Each loop can also be used with associative arrays. In this case, the syntax is slightly different:

```
for each ($array as $key => $value) {
    // Code to be executed for each element in the array
}
```

- **Explanation** In this syntax, the key is the key of the current element in the associative array, and the key is the key of the current element in the associative array, and the value is the value of the current element. Run the above code in your editor for a better and clear explanation.

Let's take an example of an associative array of prices and use the For-Each loop to iterate over the elements of the array.

1.3 CONTROL LOOPS

1.3.5: For-Each loop Example 2:

```php
$prices = array("Apple" => 0.50, "Banana" => 0.25, "Orange" => 0.75, "Grapes" => 1.00);

foreach ($prices as $fruit => $price) {
    echo $fruit . " costs $" . $price . "<br>";
}
```

Output

```
Apple costs $0.5
Banana costs $0.25
Orange costs $0.75
Grapes costs $1
```

- **Explanation** In this example, we have created an associative array of prices and used the For-Each loop to iterate over the elements of the array. The loop will execute once for each element in the array, and the variables fruit and fruit and price will hold the key and value of each element as it iterates over the array. The code inside the loop will then echo the value of the fruit variable and the value of the fruit variable and the value of the price variable, separated by the string "costs $" and add a line break after each fruit.

- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.6: While Loop:

- A while loop is a control structure in PHP that allows you to repeatedly execute a block of code as long as a specified condition is true. The syntax for a while loop is as follows:

```
while (condition) {
   // code to be executed
}
```

- **Explanation** The condition is checked before each iteration of the loop. If the condition is true, the code inside the loop is executed. This continues until the condition becomes false, at which point the loop terminates and control is passed to the code following the loop. Run the above code in your editor for a better and clear explanation.

- Here's an example of a simple while loop in PHP that counts from 1 to 5:

1.3 CONTROL LOOPS

1.3.6: While Loop:

- Here's an example of a simple while loop in PHP that counts from 1 to 5:

```
$num = 1;
while ($num <= 5) {
    echo $num . " ";
    $num++;
}
```

Output

```
1 2 3 4 5
```

- **Explanation** In this example, the condition is num<=5. As long as num<=5. As long as num is less than or equal to 5, the loop will continue to execute. Inside the loop, we echo the value of num and then increment it by1using the num and then increment itby1using the num++ shorthand notation. This continues until $num is no longer less than or equal to 5.

- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.7: Do-While Loop:

- **Do-while loop** A Do-While loop in PHP is a type of loop that allows you to execute a block of code repeatedly while a certain condition remains true. The key difference between a Do-While loop and a While loop is that the Do-While loop in php will always execute the code block at least once, regardless of whether the condition is initially true or false.

- Here's the syntax for a Do-While loop in PHP:

```
do {
   // code to be executed
} while (condition);
```

1.3 CONTROL LOOPS

1.3.7: Do-While Loop:

- **Parameters of the syntax:**
  - The "do" keyword starts the loop and indicates the beginning of the code block to be executed.
  - The code block is contained within curly braces {} and can contain any number of statements.
  - The "while" keyword is followed by a condition that will be evaluated at the end of each iteration of the loop.
  - If the condition is true, the loop will continue to execute, starting again at the beginning of the code block.
  - If the condition is false, the loop will exit and the program will continue executing the next statement after the loop.
- The key difference between a do-while loop and a while loop is that a do-while loop will always execute the code in the block at least once, even if the condition is false from the beginning. In contrast, a while loop will not execute the block of code at all if the condition is false from the beginning.

1.3 CONTROL LOOPS

1.3.7: Do-While Loop Example:

- Here's an example of a Do-While loop in PHP that will output the numbers 1 to 5:

```
$num = 1;

do {
    echo $num;
    $num++;
} while ($num <= 5);
```

- In this example, the loop will continue executing as long as the value of I is less than or equal to 5. The loop will print the value of I is less than or equal to 5. The loop will print the value of i on each iteration and then increment the value of I by1. The loop will exit when the value of I by1. The loop will exit when the value of i is greater than 5.

- Run the above code in your editor for a better and clear explanation.

1.3 CONTROL LOOPS

1.3.7: Do-While Loop Example:

- Here's an example of a Do-While loop in PHP that will output the numbers 1 to 5:

```
$num = 1;

do {
    echo $num;
    $num++;
} while ($num <= 5);
```

- In this example, the loop will continue executing as long as the value of I is less than or equal to 5. The loop will print the value of I is less than or equal to 5. The loop will print the value of i on each iteration and then increment the value of I by1. The loop will exit when the value of I by1. The loop will exit when the value of i is greater than 5.

- Run the above code in your editor for a better and clear explanation.

CONCLUSION

- Control structures in PHP are used to direct the flow of a PHP script based on certain conditions or criteria.
- PHP provides several types of control structures, including if-else statements, loops, and switch statements.
- If-else statements are used to perform actions based on whether a certain condition is true or false.
- Loops allow you to repeat a block of code multiple times, either for a fixed number of iterations or until a certain condition is met.
- Switch statements are useful for performing different actions based on the value of a variable.

PART 3

LOGICAL OPERATORS IN PHP

USE OF LOGICAL OPERATORS IN PHP

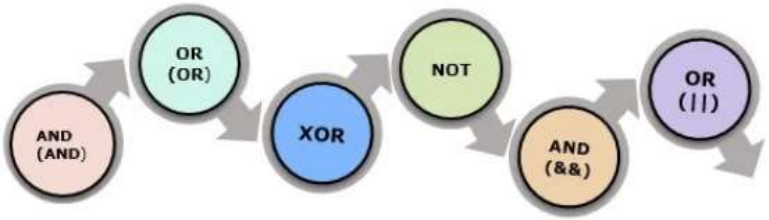
INTRODUCTION:

- PHP operators are symbols that help in doing logical operations with ease. The code generated with these operators helps in performing some specific actions.
- The logical operators include operators like addition (+), greater than (>), etc., which instruct the compiler to perform the necessary operation.
- It can check multiple operations and determine which conditions are true.
- The values which are particular operators use are known as operands.
- The operators are not similar to functions, though there can be cases where they can be used as functions.

USE OF LOGICAL OPERATORS IN PHP

LOGICAL OPERATORS IN PHP:

- Let us now look into Logical Operators in detail. We have six kinds of logical operators. They are as below:



- PHP also has logical operators that help in combining conditional statements. To name a few of these, they are AND, OR, NOT, etc.

Operator	Name	Example	Result
and	And	$x and $y	True if both $x and $y are true
or	Or	$x or $y	True if either $x or $y is true
xor	Xor	$x xor $y	True if either $x or $y is true, but not both
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true
!	Not	!$x	True if $x is not true

USE OF LOGICAL OPERATORS IN PHP

1. AND (AND) OPERATOR

The AND operator returns true if both variables being compared are true.

```php
<?php
$x = 100;
$y = 50;
if ($x == 100 and $y == 50) {
echo "AND is true";
}
?>
```

Output:

```
AND is true
```

From the example, if both conditions are not satisfied or False, the results will be False or 0. The result will return true only if all the conditions are true.

USE OF LOGICAL OPERATORS IN PHP

2. OR (OR)

Similarly, the OR operator works if either of the conditions is true.

```php
<?php
$x = 100;
$y = 50;
if ($x == 100 or $y == 80) {
echo "XOR is TRUE";
}
?>
```

Output: `XOR is TRUE`

From the example, if any of the conditions is true, the results will be True. The result will return false only if all the conditions are false.

USE OF LOGICAL OPERATORS IN PHP

3. XOR

The XOR condition returns true when either or the variables are true, not both are true.

```php
<?php
$x = 100;
$y = 50;
if ($x == 100 or $y == 80) {
echo "XOR is TRUE";
}
?>
```

Output: XOR here!

From the example, if any of the conditions is true, the results will be True. The result will return false if all true or false.

USE OF LOGICAL OPERATORS IN PHP

4. NOT

The NOT operator is used when it is required to check if a particular variable is not true. This means we can use NOT when we have to check if any condition is not true.

```php
<?php
$x = 100;
if ($x !== 90) {
echo "NOT is here";
}
?>
```

Output: `NOT is here`

In this example, you can see that we are checking if the variable is not 90. The variable x is 100 and which satisfies the NOT condition. Due to this, we have the output following the specified condition; hence, you see the output as 'NOT is here.'

USE OF LOGICAL OPERATORS IN PHP

5. AND &&

This is similar to the AND that we saw previously. It will return the value as true only when both conditions are true or when both variables evaluate to be true.

```php
<?php
$x = 100;
$y = 50;
if ($x == 100 && $y == 50) {
echo "&& is true!";
}
?>
```

Output:

```
&& is true!
```

USE OF LOGICAL OPERATORS IN PHP

6. OR ||

Similar lines, the OR condition is also the same as the OR mentioned above. This operator works even when one of the specified conditions is true. It has similar results, just like the tap example mentioned previously. OR having three different sinks can fill the sink even when only one tap is open.

```php
<?php
$x = 100;
$y = 50;
}
if ($x == 100 || $y == 80) {
echo "OR is true!";
}
?>
```

Output: OR is true!

In the above example, the variable x satisfies the condition specified for $x=100, and hence the result displays the message for when the result is true. Though the condition for variable y is not satisfied, the output is displayed. This is because of the OR condition, which works even if one condition is being satisfied.

CONCLUSION

- PHP has many logical operators, which make it easy to use.
- The PHP compiler helps in compiling these operators fast.
- The logical operators help in performing logical operations.
- These can be arithmetic, logical, string, or array operations.
- PHP has a facility for performing all these operations.
- It helps in checking multiple conditions at one time.
- This saves time and increases the optimization of the PHP compiler.
- Therefore, it is advisable to utilize these operators when working with PHP. Logical operators expedite the execution of logical operations, ensuring quick results.
- These conditions thus help you to get Boolean results and work on them accordingly.

PART 4

BEST PRACTICES FOR CODE ORGANIZATION AND READABILITY IN PHP

Best Practices for Code Organization and Readability in PHP

- This lecture attempts to teach you with some of the best practices of one of the most widely used programming languages in the world, PHP.
- There are many beginners or even experienced PHP developers who don't bother to follow the best practices of the language, either unintentionally or intentionally.
- Quite often, many people loose many high-paying jobs just because they have never put their attention to the best practices of the language. And it's really very difficult for anybody to get hold of the best practices overnight.
- So, if you want to be a professional developer then you must know, understand and follow the best practices of the language and should start practicing them from the very next piece of code you write.

Best Practices for Code Organization and Readability in PHP Continues…

1) Maintaining a Proper Documentation of your Code

- Maintaining a proper documentation of your code is more important than you had ever thought of. It's really very unfortunate seeing some beginners or even seasoned programmers not focusing on writing meaningful comments to their codes just because they are too lazy.
- Not only does it help others to understand your code better, but also it helps you to remember what you have written when you review it in the future.
- Suppose today, you write a critical function and you remember how it works for a day or two or even for a week. But there is an ample scope for you to forget what it does, a month or two later when you review it.
- So, it is always a good practice to write proper comments to make your code understandable to everybody (even yourself).

Best Practices for Code Organization and Readability in PHP Continues…

2) Maintaining a Proper Coding Standard

- It is very essential to maintain a proper coding standard, as it can become a painful problem if different programmers (working on a same project) starts using different coding standards.
- The source code can become completely unmanageable, if everybody working on them starts inventing their own coding standards.
- There are some programmers who don't even bother to maintain any coding standards at all and their code starts looking like a huge pile of garbage and nothing more than that.
- If you maintain a proper coding standard, then it would be easier for others to debug your code and your joint projects will tend to be much more productive.

Best Practices for Code Organization and Readability in PHP Continues…

3) Never Use Short Tags

- There are many programmers, who tries to take a shortcut way of declaring PHP. Using **"<?"** or **"<%"** is never a good practice and it doesn't make your code look more professional than others
- This simple malpractice can cause conflicts with XML parsers and can also make your code incompatible with future versions of PHP.

Best Practices for Code Organization and Readability in PHP Continues…

4) Use Meaningful Variable and Function Names in your Code

- It can surely become a painful experience for other programmers to understand your code if you don't stick with a proper naming standard and use generic and un-meaningful names everywhere. It's not a good practice at all, and you must not follow it in any way.
- Always try to use meaningful and grammatically sensible names for your variables and functions and try to make the good habit of separating every word with underscores. And also try to be consistent with the standard you are following so that other people can get to understand your convention quickly and easily.

Best Practices for Code Organization and Readability in PHP Continues…

5) Indentation, White Spacing and Line Length

- As already mentioned the importance of maintaining a proper coding standard, but the things that need to be specifically considered are indentation, white spaces and line lengths.
- Try to keep an indent of 4 spaces. Never use Tab as different computers can have different Tab settings. Also try to keep the line length less than 80 characters to ensure that the code is easily readable to you and other developers.
- The main idea here is to make the code look clean and easily readable and debuggable by you and other fellow programmers.

Best Practices for Code Organization and Readability in PHP Continues…

6) Single Quoted vs. Doubles Quoted Strings

- You have to understand the difference between the single quoted strings and the double quoted ones. If you just have a simple string to display, then always go for single quotes.
- But if you are going to have variables and special characters like “n”, “t” in your string, then you must use double quotes which will get your string parsed by the PHP interpreter and will take more execution time than single quoted strings.
- So, try to understand the difference in their working nature and use them appropriately, whenever necessary.

Best Practices for Code Organization and Readability in PHP Continues…

7) Never Use Functions inside Loops

- I have seen many programmers who tend to make the mistake of using functions inside of loops. If you are doing this intentionally and is ready to compromise performance just for the sake of saving a line of code, then you certainly need to think once again.
- **Bad Practice:**

```
for ($i  = 0, $i <= count($array);  $i++){
//statements
}
```

- **Good Practice:**

```
$count = count($array);
for ($i = 0; $i < $count;  $i++){
//statements
}
```

- If you take the burden of storing the value returned by the function in a separate variable before the loop, then you would be saving enough execution time as in the first case the function will be called and executed every time the loop runs which can increase the time complexity of the program for large loops.

Best Practices for Code Organization and Readability in PHP Continues…

8) Using Single Quotes around Array Indexes

- There is a difference between $array['quotes'] and $array[quotes] and you have to understand this difference properly.
- A feature of PHP is that, it considers unquoted strings used as array indexes as constants, and if the constant have not been defined before, then it will be "self-defined" and a warning will be raised.
- This will not stop your code from working, but the bug will remain in the code.

Best Practices for Code Organization and Readability in PHP Continues…

9) Understanding Strings in a Better Way

- Take the code snippet as an example and try to guess which statement will run the fastest and which one the slowest.
- **Code Snippet:**

```
$a = 'PHP';
print "This is my first $a program.";
echo "This is my first $a program.";
echo "This is my first ".$a." program.";
echo "This is my first ",$a," program.";
```

- Guess what, the last and the most uncommon statement of all wins the speed test. The first one obviously loses as the "print" statement is slower than "echo" statement. The third statement wins over the second one as it uses concatenation operation rather than using the variables inline.
- The lesser-known last statement wins as there are no string operations being performed and is nothing other than a list of comma-separated strings.

Best Practices for Code Organization and Readability in PHP Continues…

10) Using the DRY Approach

- DRY or Don't Repeat Yourself is a programming concept in software engineering, which tries to cut down redundancy from your code.
- It's not a PHP-specific concept and can be applied to any programming language like JAVA, C++, etc. An example code will help you understand the DRY approach in a much better way.

```
$mysql = mysql_connect('localhost', 'admin', 'admin_pass');
mysql_select_db('wordpress' or die('Cannot Select Database.');
```

- Now after using the DRY approach, the code will look something like this:

```
$db_host  = 'localhost';
$db_user = 'admin';
$db_pass = 'admin_pass';
$db_name = 'wordpress';
$mysql = mysql($db_host, $db_user, $db_pass);
mysql_select_db($db_name);
```

Best Practices for Code Organization and Readability in PHP Continues…

11) Try to Prevent Deep Nesting

- Try to prevent deep nesting levels in your code, as much as possible. It can make things really difficult for you when you need to debug your code and can take the heck out of people who will try to review your code.

- Try to use conditions as logically as possible to avoid unnecessary deep nesting. It is not only a very poor programming practice, but also can make your code look ugly enough to your fellow developers.

Best Practices for Code Organization and Readability in PHP Continues…

12) Make the PHP Manual Your Best Friend

- The best thing about PHP is that it has got a very well-documented manual for you to use anytime you need. Just head over to *http://php.net* and you will get almost every possible information you may need, in a very well-organised manner.

- They have also got a new look to their site (which is still in beta stage), for you to try out. Another great thing about the PHP manual is that almost every article is filled with some truly useful comments.

Best Practices for Code Organization and Readability in PHP Continues…

13) Time to Use an IDE

- IDE stands for Integrated Development Environment which can make PHP development a lot easier and interesting for developers. IDE's offer a great range of features like *Syntax Highlighting, Code Completion, Navigation, Documentation, Debugging*, etc. which can make you much more productive and can help you to write good code with very less mistakes.

- Some of the best IDE's for PHP are VSCode, NetBeans, phpDesigner, phpStorm, PHP Introduction and Resources, etc. Choose the one which suits you the best.

Best Practices for Code Organization and Readability in PHP Continues…

14) Try a PHP Framework

- If you have learned the fundamentals of PHP then it's time for you to try some PHP frameworks. There are tons of PHP frameworks available out there, most of which are designed on the basis of Model-View Controller (MVC) software architecture.

- There are lots and lots of new and interesting things for you to learn using a PHP framework. Frameworks like Laravel, CakePHP, CodeIgniter, Yii, Zend, Symfony etc can help you create some awesome PHP applications with ease.

THANK YOU